Architecture interne des systèmes d'exploitation

Projet « ExPResSO »

BAUER Théo - M1 CHPS

1 Introduction

La librairie ExPResSO permet de réaliser de la parrallélisation à un haut niveau de programmation. Son fonctionnement se base sur la répartition de taches d'une file d'attente, à exécuter dans des fils d'exécution enfants en mémoire partagée. Pour cela nous utilisons des appels aux fonctions "pthread_barrier_wait", "pthread_cond_wait", "pthread_cond_broadcast", "pthread_mutex_lock" et "pthread_mutex_unlock" de la librairie "pthread". Le principe est donc d'établir une communication entre le fil d'exécution parent (ou fil d'exécution principale) et les fils d'exécution enfants. En effet, broadcast envoi un signal à tout les fils d'exécutions du programme, que nous récuperons à condition de se trouver dans un "wait". Deux cas de figure se présentes pendant l'exécution du code :

— Tout les fils d'exécution enfants sont en attente d'un signal du parent puis le parent envoi le broadcast, intercepté par tout les enfants leurs permettant de continuer leurs exécutions. Pendant le "wait" les ressources utilisées du processeur sont moindres.
— Tout les fils d'exécution (parent+enfants) attendent à une barrière, qui se débloque uniquement lorsqu'ils ont tous atteint cette barrière.

2 Version de base

Le document expresso.c contient l'implémentation de toutes les fonctionnalitées de la librairie. Nous décrivons dans cette partie ce que fait chacune des fonctions existantes.

2.1 Variables globales

ExPResSO utilise des fils d'exécution en mémoire partagée, c'est pourquoi les variables sont globales afin de pouvoir y accéder dans chacune des fonctions et dans chacun des fils d'exécution. En particulier nous définissons des structures de données qui nous servirons tels que les arguments passés dans les fils d'exécution enfants ou une structure contenant les informations des taches à effectuer.

Nous retrouvons également la fonction "void * set_child_task(void * arg)" qui est la partie de code executée par chaque fil d'exécution enfant (voir expresso_wait [2.9]).

2.2 expresso_worker_count

Lit et renvoi la variable d'environnement donnant le nombre de fils d'exécution à créer pour le parrallélisme.

2.3 expresso_wall_time

Renvoi simplement un temps à l'ordre des nanosecondes.

2.4 expresso_schedule_set

Modifie la variable globale "my_policy" selon la politique d'ordonnancement [5] donnée en argument.

2.5 expresso_schedule_get

Permet de récuperer la politique d'ordonnancement actuelle.

2.6 expresso_initialize

Afin d'utiliser toutes les fonctions d'ExPResSO qui suivent, nous devont obligatoirement faire appel à cette fonction. Celle-ci alloue la mémoire pour les tableaux, initialise toutes les valeurs des variables globales, puis crée les fils d'exécution enfants tout en s'assurant qu'ils soient prêts à exécuter des taches, en effet le temps de création des enfants n'est pas déterministe donc nous nous assurons que tout les enfants aient commencés à exécuter du code grâce à une barrière. Une fois les enfants créés, ils se retrouvent dans une boucle infinie de laquelle ils sortiront grâce à expresso_finalize, puis vont pour l'instant dans un wait de la condition "init".

2.7 expresso_task

Ajoute une tache dans un tableau "task_info->my_tasks" afin d'avoir à une position donnée la fonction et ses arguments. Une condition sur la taille maximale de ce tableau à été ajoutée car nous avons besoin d'allouer la mémoire avant de connaître le nombre maximum de taches qui seront exécutées.

2.8 expresso_weighted_task

Similaire à expresso_task, mais attribue également la donnée du poids associée à une tache, ne s'appelle que si la politique d'ordonnancement est "SCHEDULE_BALANCED" (voir politique d'ordonnancement [5]).

2.9 expresso_wait

Nous allons ici expliquer le fonctionnement de la parrallélisation du programme :

Enfants : en attente de "init" ;

Parent : initialise les données nécessaires, broadcast "init" puis se met dans une barrière.

Enfants : commencent à exécuter les calculs selon la file d'attente remplis par les appels à expresso_task déjà réalisés et selon la politique d'ordonnancement. Vu que chaque tache est à une position donnée d'un tableau, nous n'avons pas à utiliser de verrou pour les effectuer, cependant nous devons nous assurer d'utiliser les verrous lors de l'incrémentation des compteurs stockées en tant que variables globale. Lorsque les enfants finissent les taches, ils sortent de leurs boucles while et atteigne la même barrière que le parent, et lorsqu'ils se débloquent attendent un signal "re_init" ;

Parent : modifie les données nécessaires pour la réintialisation, puis broadcast "re_init" et rencontre une barrière à nouveau ;

Enfants : comme pour leurs création, ils assurent être prêts pour un nouvel appel à expresso_wait en rencontrant la barrière, puis en entrant dans un wait de "init" ;

Fin de expresso_wait.

2.10 expresso_stats

Affiche sur la sortie standard :

- Le nombre de taches exécutées par chaque enfant
- Le nombre total de taches exécutées par tous les enfants
- La moyenne du nombre de taches exécutées par chaque enfant

2.11 expresso_finalize

Il est important de toujours appeler cette fonction si un appel a expresso_initialize à été fait.

Cette fonction change une donnée appelée "terminate" puis envoi un broadcast de "init", les enfants plutôt que de chercher à exécuter des taches, vont rentrer dans un "if" dépendant de terminate et

sortir de leurs boucle infinie, finalisant donc leurs exécutions. Le parent fait un appel à join, qui pour chaque enfant attend que celui-ci finisse puis le join proprement.

Par la suite il suffit de détruire les conditions, barrière et mutex, puis libérer la mémoire allouée précédemment dans expresso_initialize.

3 Expresso-benchmarks

Quatre programmes ont été parrallelisés grâce à ExPResSO, et nous avons comparé le temps d'exécution des calculs de chacun d'entre eux avec une implémentation séquentielle. Nous expliquons dans cette partie les choix algorithmiques en terme de parrallélisation.

Puisque expresso_task prend en argument une fonction et ses paramètres selon une signature particulière, nous créons si besoin au début de chaque programme une fonction et une structure de données adaptés à l'utilisation de ExPResSO. De plus, dans chaque programme nous avons gérer le cas où nous utilisons ou pas la politique d'ordonnancement [5] "SCHEDULE_BALANCED" auquel cas nous donnons un poids adapté aux taches.

3.1 dot.c

Chaque tache correspond au calcul du dot product sur un block de taille donnée. Nous initialisons les données passées en arguments de chaque thread, comportant en particulier la position de début et de fin de chaque tache dans les vecteurs.

Une fois les taches initialisées nous appelons les fonctions de ExPResSO tel que prévu, nous ajoutons les taches à la file d'attente, le poids est égale au nombre d'éléments dans chaque block, et faisons appel à expresso_wait afin de réaliser les calculs des multiplications en parrallèle. La somme des résultats de chaque block se fait ici en séquentielle car "sum_expresso" est une variable globale.

3.2 list.c

Ce programme incrémente simplement les éléments d'une liste de 1, mais effectue une attente en temps dépendante de la position de l'élément incrementé. La parrallélisation de ce programme est naïf, chaque élément de la liste est une tache, de poids sa position.

3.3 vecmul.c

Similairement à dot.c, nous réalisation la multiplication des vecteurs par block, nous faisons correspondre le pointeur "list" à "expresso" puis le passons en argument de "vecmul" exécutée par les enfants qui est ici déjà adaptée pour notre librairie. Le poids des taches est égale au nombre d'éléments par block.

3.4 matmul.c

Nous utilisons la variable "row_block" pour déterminer le nombre de taches à réaliser, nous stockons les données modifiées par les enfants dans l'argument "data.c" de notre structure de données. Visuellement, dans un exemple simple, nous parrallélisons ce programme de la forme suivante : pour deux matrices

ayant pour dimensions $(n, m) = (3, 10)$ et $row_block = 4$

$$\left(\begin{array}{cccc|cccc|cc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \hline 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ \hline 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \end{array}\right) \times \left(\begin{array}{c|c|c} 11 & 10 & 9 \\ 10 & 9 & 8 \\ 9 & 8 & 7 \\ 8 & 7 & 6 \\ \hline 7 & 6 & 5 \\ 6 & 5 & 4 \\ 5 & 4 & 3 \\ 4 & 3 & 2 \\ \hline 3 & 2 & 1 \\ 2 & 1 & 0 \end{array}\right)$$

Pour ce faire, nous effectuons la parrallélisation sur la boucle la plus externe de la multiplication de matrices. Remarquons également que dans l'exemple ci-dessus, row_block n'est pas multiple de n et nous avons correctement gérer cela dans le code, ce qui a son importance également pour le poids des taches car il est égale au nombre d'éléments par block, soit ici le block sur la première ligne de la première matrice contenant "1 2 3 4" à un poids égale à 4, tandis que le block "8 9" à un poids égale à 2.

4 Performances des applications parrallélisées

À part pour list.c les applications ne sont pas plus rapides que l'implémentation séquentielle. Je pense que la raison est du fait que l'initialisation et la fin des taches peut prendre du temps que l'implémentation séquentielle n'as pas a faire. Cependant cela implique que la comparaison du temps d'exécution dépend de la taille du problème : j'ai donc testé matmul.c pour des matrices de dimensions $(n, m, k) = (1024, 1024, 1024)$ et observe le résultat suivant pour chacune des politique d'ordonnancement [5] :

Implementing matmul
n = 1024, m = 1024, k = 1024
dynamic
Total tasks executed : 131072
Tasks executed by child 0 : 130976
Tasks executed by child 1 : 32
Tasks executed by child 2 : 32
Tasks executed by child 3 : 32
Mean of tasks executed by children : 32768.000000
Reference computation time : 13.055 s
ExPResSO computation time : 5.67014 s
Good results :-)
Implementing matmul
n = 1024, m = 1024, k = 1024
static
Total tasks executed : 131072
Tasks executed by child 0 : 32768
Tasks executed by child 1 : 32768
Tasks executed by child 2 : 32768
Tasks executed by child 3 : 32768
Mean of tasks executed by children : 32768.000000
Reference computation time : 13.0283 s
ExPResSO computation time : 5.86404 s
Good results :-)
Implementing matmul

n = 1024, m = 1024, k = 1024
balanced
Total tasks executed : 146
Tasks executed by child 0 : 36
Tasks executed by child 1 : 34
Tasks executed by child 2 : 35
Tasks executed by child 3 : 41
Mean of tasks executed by children : 36.500000
Reference computation time : 13.1367 s
ExPResSO computation time : 5.88257 s
Good results :-)

Remarque : pour ce test j'ai du augmenter "max_tasks" dans le fichier expresso.c, expresso_initialize prenait plus de temps à s'exécuter car l'allouement de mémoire était devenu lourd.

Nous en concluons donc que pour des problèmes suffisament grands, l'utilisation de ExPResSO devient pertinente et la parrallélisation s'effectue correctement selon la politique d'ordonnancement, cependant je ne doute pas que des éventuelles améliorations de performances soient possibles sur la librairie mais suis satisfait de la version actuelle et ne vois pas comment obtenir de grosses améliorations.

5 Version avancée et politique d'ordonnancement

Nous avons le choix entre trois politiques d'ordonnancement : dynamic, static et balanced. Les trois sont présentes dans la fonction set_child_task de expresso.c grâce à un switch, et exécutent les taches de façon differentes :

- Dynamic : la politique par défaut de la librairie, les enfants effectuent une tache dès qu'elle est disponible jusqu'à ce que toutes les taches soient réalisées. Nous avons besoin d'un verrou pour incrémenter le compteur de tache executée et la position de la tache effectuée.
- Static : réparti les taches équitablement entre les enfants, nous pouvons le faire dans une boucle "for" plutôt qu'un "while", ce qui nous permet notamment d'éviter d'avoir un verrou dans la boucle, les compteurs ajoutent donc le nombre d'itérations effectuées dans le "for" à la suite de celui-ci.
- Balanced : Un poids est associé à chaque tache et nous mettons les taches dans la file d'attente grâce à la fonction expresso_weighted_task. Dans cette fonction calcul la somme de tout les poids, nous pouvons donc le diviser par le nombre d'enfants pour avoir la moyenne des poids à répartir par enfant. Lorsque la somme des poids de chaque taches executées par un enfant est supérieur à cette moyenne, l'enfant n'effectue plus aucune tache, sortant donc du switch. Cela permet bien de répartir les taches équitablement entre les enfants, mais implique que l'utilisateur de la librairie ExPResSO doit pouvoir estimer le poids des taches qu'il doit effectuer.

Sur les applications parrallelisées, il n'y a pas de différence majeure sur les temps d'exécution, mais nous observons tout de même que la politique dynamic a tendance à assigner beaucoup de tache à un unique enfant, nous pouvons remédier à cela en ajoutant un "sleep" très petit dans le code des enfants, mais je ne trouve pas que ca vaille la peine de ralentir le code intentionnellement.

Pour une parrallélisation de taches plus variées, c'est-à-dire pas forcément pour une même fonction avec une même variable ou un même poids entre les taches, utiliser ces différentes politiques pourraient montrer une différence plus importante en temps d'exécution, et nous pouvons adapter la parrallélisation d'une application grâce à l'une des politiques selon les besoins de l'utilisateur.